# Lecture Session

## Requirements

- What is a requirement?
- Functional and non-functional requirements
- Requirement measure

# What is a requirement?

- A requirement is a feature that your system must have in order to satisfy the customer.
- What the system should do.
- It may range from a high-level abstract statement of a service or a low-level detailed specific system constraint.

# Who provides requirements?

1. Customer
2. End user
3. Development team members
4. Management
5. Technology providers

We call these people **STAKEHOLDERS**.

```
We create software for a reason.
We create software for people.
We need to know what the people want in the software.

These are called REQUIREMENTS.
```

# Stakeholder

- Any person or organization who is affected by the system in some way and so who has a legitimate interest
- Stakeholders view the system differently
  - perspective
  - priorities

NOTE: A system may not always have an identifiable stakeholder who sets the requirements. Some software is developed according to an organisation's perception of market demand.

# Example: A Customer Relationship Management (CRM) System

Sales Team:
Priority: High
Function: Lead Management and Sales Pipeline Tracking

IT Department:
Priority: Medium
Function: System Integration and Security

Marketing Team:
Priority: Medium
Function: Customer Segmentation and Campaign Analytics

Management Team:
Priority: Varies
Function: Comprehensive Reporting and Analytics

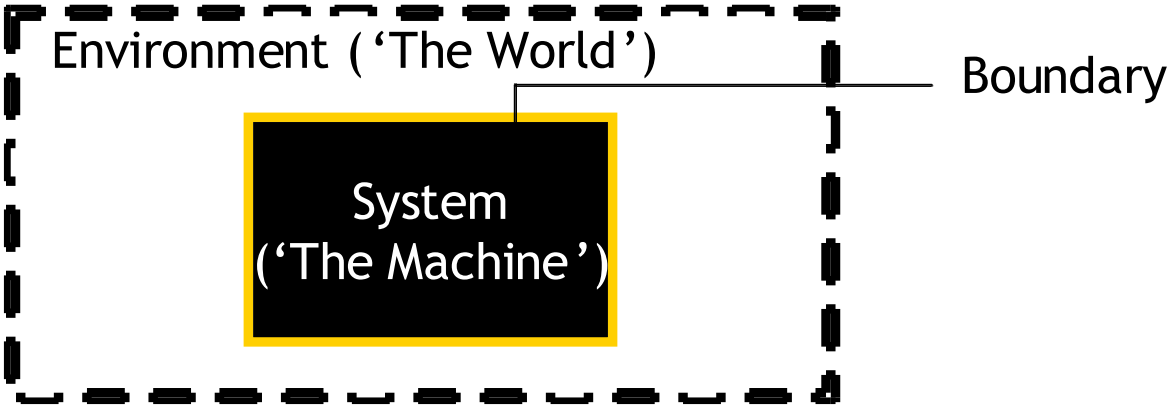
Customer Support Team:
Priority: High
Function: Ticketing System and Customer Communication

End Users (Sales Representatives):
Priority: Medium
Function: User-Friendly Interface and Mobile Accessibility

# Identify the requirements

- Talk to the stakeholders
- Ask questions
- Brainstorm
- Look at the problem from many points of view

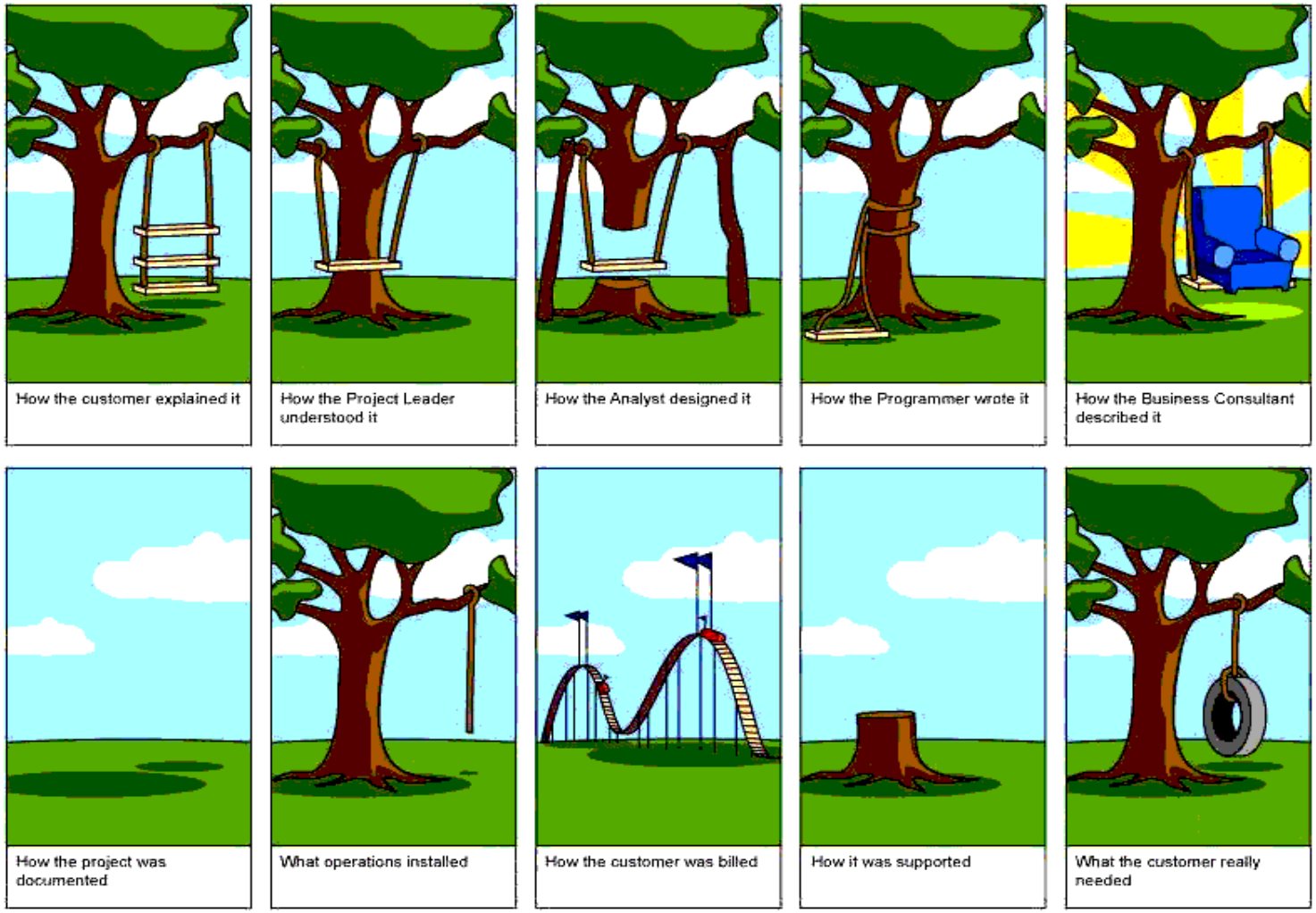
Not easy!



# The difficulty of identifying requirements

- Understanding the requirements is among the most difficult tasks that face a software engineer:
  - Does the customer know what is required?
  - Does the end-user have a good understanding of the features and functions that will provide benefit?
  - What they said = what they mean?
  - Good communication?
  - Even if the answers are all yes: requirements will change, and will continue to change throughout the project!

# The classic “The Tire Swing” joke




http://projectcartoon.com/

# Why are requirements important?

"The hardest single part of building a software system is deciding precisely what to build. No other part of the conceptual work is as difficult as establishing the detailed technical requirements, including all the interfaces to people, to machines, & to other software systems. No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later."

*Fred Brooks in "The Mythical Man Month"*

# The Economics of Requirements

Relative cost to fix an error:

| Phase in which found | Cost ratio |
|---|---|
| requirements | 1 |
| design | 3-6 |
| coding | 10 |
| development testing | 15-40 |
| acceptance testing | 30-70 |
| operation | 40-1000 |

… and these figures are considered conservative!

# Requirements Engineering

- “Solution” of those challenges.
- Helps software engineers to better understand the problem they will work to solve.
- Software engineers and other stakeholders all participate in requirement engineering.
- The extra time invested will save time and money in the long term.

# Requirement classification

- Functional requirements
  - Services
- Non-functional requirements
  - Constraints: timing constraints, constraints on the development process, standards, etc.

# Functional requirements

- Describe what the system should do – Functionality. For example in a Virtual Learning Environment (VLE) system:

- The lecturer shall be able to set an assignment and its deadline.
- The student shall be able to submit coursework.
- The student shall be able to and view marks.
- The administrator shall be able to enter all the module information.
- The administrator shall be able to create a student list.
- The student shall receive a notification of the announcement.
- The lecturer shall be able to provide feedback comments.

# Identify Functional requirements

- Should have …
  - Completeness: include descriptions of all facilities required.
  - Consistency: no conflicts or contradictions in the descriptions of the system facilities.
- Difficulties
  - Problems arise when requirements are not precisely stated.
  - Ambiguous requirements may be interpreted in different ways by developers and users.

# Non-functional requirements

- Define system properties and constraints
  - Reliability, response time, storage requirements.
  - I/O device capability, system representations, etc.
- Process requirements
  - Quality standard, programming language, development method, etc.
- Non-functional requirements may be more critical than functional requirements.
  - *If these are not met*, the system is useless.

# Non-functional examples

- Product requirements

The user interface shall be implemented using HTML5 and JavaScript.

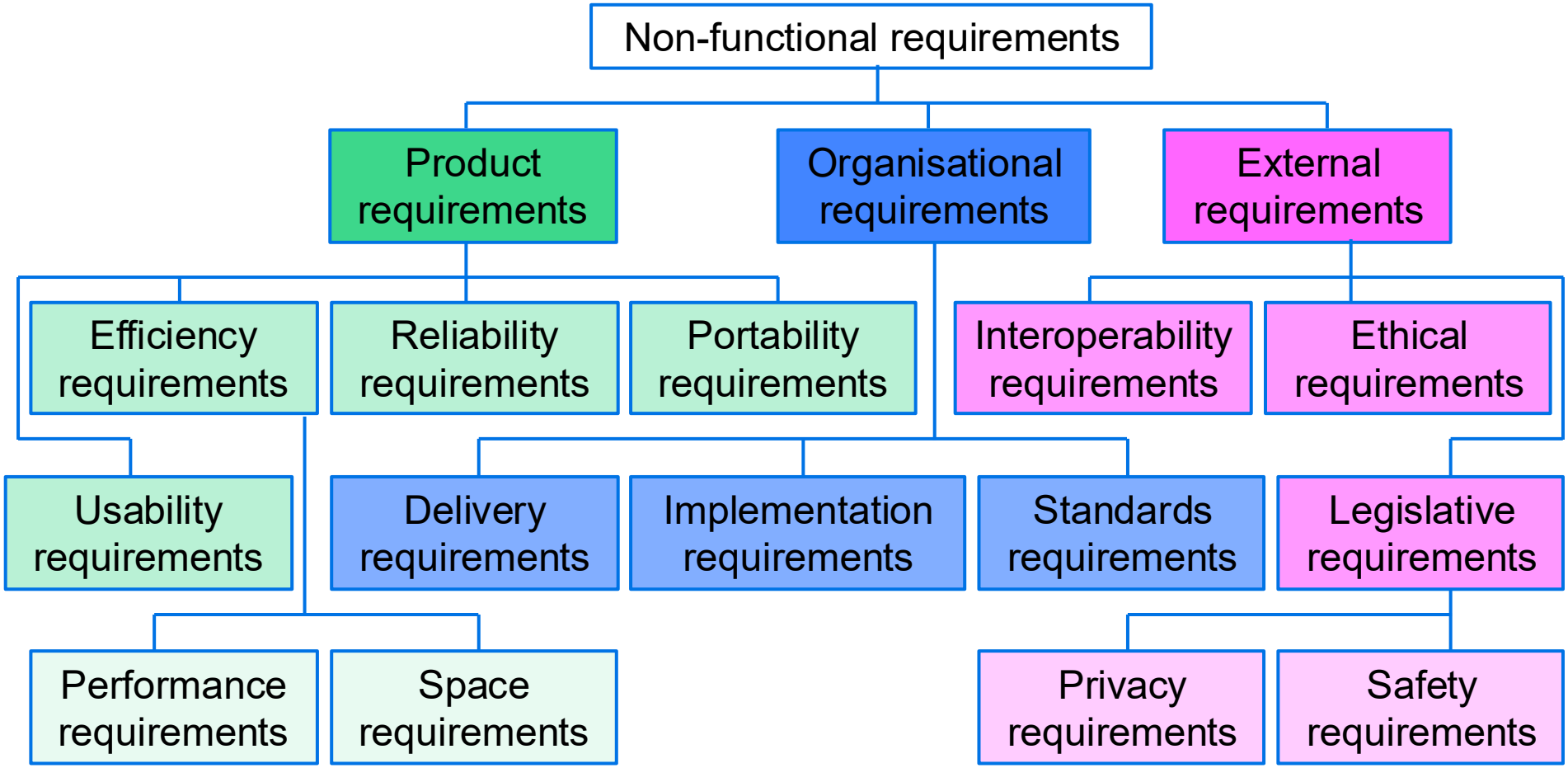
- Organisational requirements (policies and procedures)

The system development process shall conform to the process defined in the IT policy document.

- External requirements

The system shall not disclose any personal information about customers, apart from their name and reference number to the operators of the system.

# Non-functional requirements




"Software Engineering" by Ian Sommerville, Addison-Wesley

# Identify Non-functional requirements

- Be able to verify

The system shall response fast.

The system shall be lightweight.

The system shall be portable.

- Verifiable non-functional requirement
  - Using some measure that can be objectively tested.
  - Quantitative

# More Examples of metrics

- System goal

The system should be easy to use by an experienced administration staff and should be organised in such a way that user errors are minimised.

- Verifiable non-functional requirement

An experienced administration staff shall be able to use all the system functions after a total of two hours training.

# Measures

| Property | Measure |
|---|---|
| Speed | Event response time<br>Transactions/second<br>Auto refresh time |
| Size | MB/GB |
| Ease of use | Training time<br>Pages of instructions |
| Reliability | Rate of availability<br>Mean time to failure<br>Probability of unavailability<br>Rate of failure occurrence |
| Robustness | Time to restart after failure<br>Percentage of events causing failure |
| Portability | Number of target systems<br>Percentage of target dependent statements |

# Practical Exercises Session (15-20 minutes in class)

- Identify Functional and Non-functional requirements
- Non-functional requirement measure

# Lecture Session

## Requirements

- Requirements conflicts
- Requirements capture techniques

# Requirements conflicts

- Conflicts between different requirements are common.
- Example: In a digital library system

**Requirement A: Item Retrieval**

**"This system allows the user to retrieve items in any format".**

**Requirement B: No File Conversion**

**"File conversion should not be supported".**

# Requirements conflicts

*Which is the most critical requirement?* There has to be a trade-off between the requirements.

**Options as follows:**

1. Support file conversions for all major types and increase the budget for the project.
2. Support file conversions for limited set of formats (e.g. .pdf, .rtf and .doc) and increase the budget for the project.
3. Add the requirement: "A separate version of each item, one for every format required, must be submitted to the system when a new item is added".".
4. Only support the retrieval of items in formats that are available.

Choose the best option and this has to be agreed by all stakeholders.

# Requirements Capture

- Requirements capture:
  - What is to be built?
  - Different starting points for each project, related to what kind of system is required, the type of organisation that requires the system, the technology etc.

Starting point:

- Determine boundary and interfaces.
- Elaborate functions, features, behaviours.
- Identify data, data transformations.

# Fact-finding Techniques

- Fact-finding techniques
  - Background reading
  - Interviewing
  - Observation
  - Document sampling
  - Questionnaires
  - …

# 1. Background Reading

- Market research
- Competitive Analysis
- Learn more about the users/organisation: what is this software for?
- Brainstorming

**Provides a high-level overview of system components and their interactions**

# 2. Interviewing

- Direct discussions with stakeholders. Most widely used technique
  - One to one
  - Focus group
  - Workshop

**Allows for personalised and in-depth exploration of requirements**

# 3. Observation

- Directly observing stakeholders using the current system
- Observe workflow
- Observe interaction

**Provides insights into real-world scenarios and user behaviours**

# 4. Document Analysis

- Reviewing existing documents, data, manuals, and reports. A specialist form of observation.
- Ideal for capturing non-functional requirements.
- Information processing

**Aids in understanding existing data, processes and documentation.**

# 5. Questionnaires

- Economical way of gathering large amount of data, ideal for the development of systems to be used by multiple users on multiple sites.
- Results could be analysed by a computer.
- Often suffer from “questionnaire fatigue” and response is therefore low or inaccurate.
- Difficult to write a good questionnaire.

**Can efficiently gather input from a large number of participants**

# Practical Exercises Session (15-20 minutes in class)

- Resolve requirement conflict issues
- Use appropriate factfinding techniques

# Summary

- Requirements
  - What is a requirement?
  - Functional and non-functional requirements
  - Requirement measure
  - Requirements conflicts
  - Requirements capture techniques